

LEGALeASE

AI-Powered Legal Document Generator

PRE-REQUISITES & PROJECT DEVELOPMENT REPORT

Brainstorming & Ideation • Requirement Analysis • Project Design •
Project Planning • Development • Testing • Documentation •
Demonstration • Images • Full Diagrams

Project Identity
Project Type: AI-assisted web application for structured legal-document drafting.
Backend Focus: Python + FastAPI REST services.
Suggested Frontend: HTML/CSS/JavaScript or React.
Suggested AI Layer: configurable LLM service with prompt templates and validation.
Document Output: preview, structured text, and PDF/DOCX export.
Database: SQLite for prototype; PostgreSQL for production.
Audience: students, developers, legal-tech demonstrators, and supervised document-preparation workflows.

This 50-page report is designed as a project-ready pre-requisites document. It explains what must be understood, installed, designed, developed, tested, documented, and demonstrated before and during implementation. It also provides text-based full-page diagrams that can be redrawn in draw.io, PowerPoint, Visio, or any diagram editor.

Prepared for academic project submission

How to Use This Report

Section 2

This document treats LegalEase as an educational software project rather than a substitute for professional legal advice. The system should generate drafts from user-provided information, clearly label outputs as drafts, and encourage review by a qualified professional when appropriate.

The pre-requisite stage establishes the technical, functional, data, security, usability, and documentation foundations required for implementation. A strong pre-requisite report reduces rework because the team agrees on scope, interfaces, data structures, testing strategy, and expected outputs before coding.

The project can be implemented incrementally. First create the user interface and API skeleton, then add validation, document templates, AI integration, storage, export, testing, and demonstration features.

Each major section includes objectives, activities, expected outputs, risks, and acceptance ideas. The diagrams use boxes and arrows represented as text so that the report remains self-contained and printable.

The report intentionally uses a practical academic-project structure: problem understanding, requirements, architecture, planning, development, quality assurance, documentation, and demonstration.

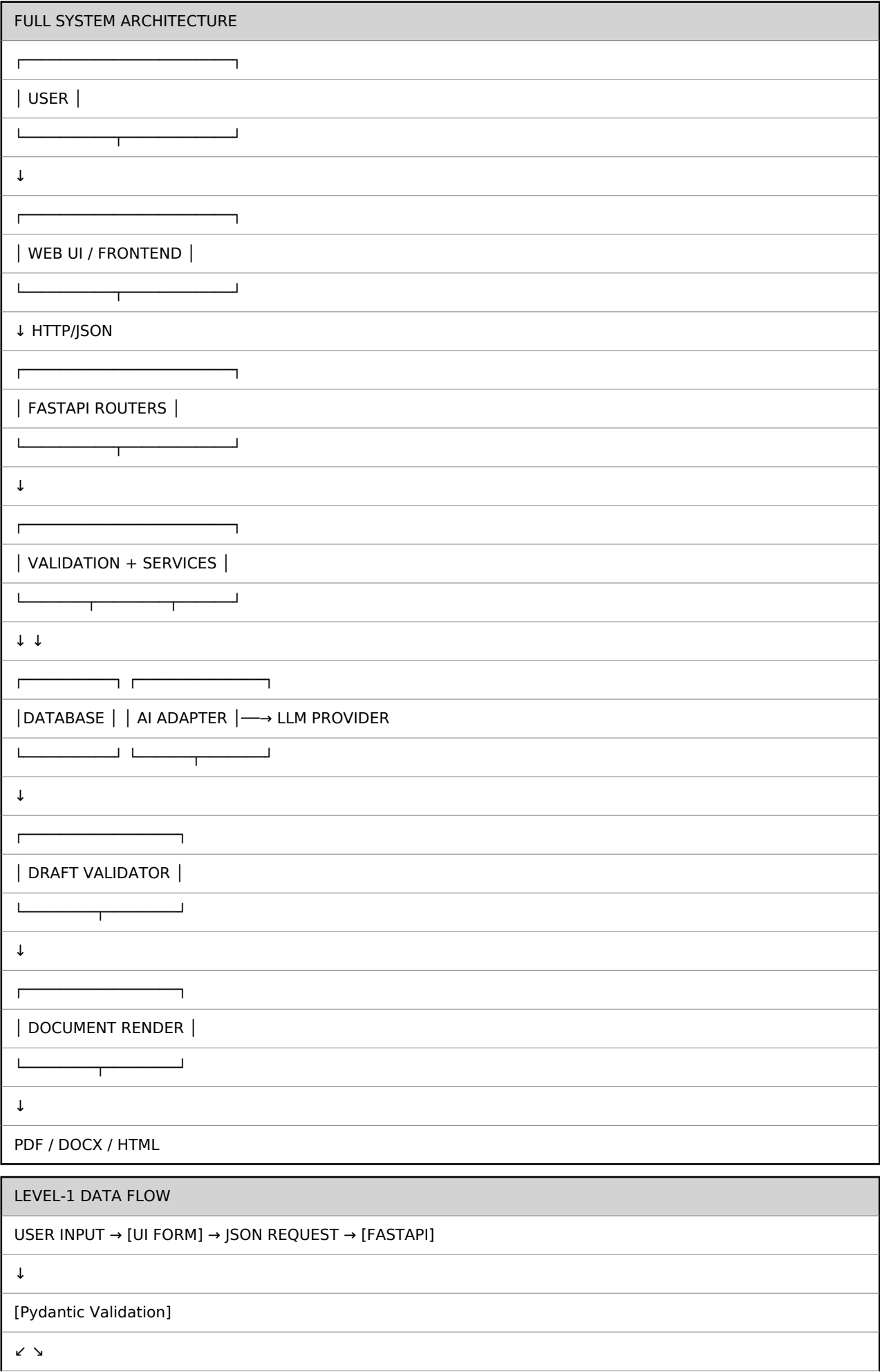
- Use the checklist pages to confirm that tools and accounts are ready.
- Use the architecture pages as the baseline for module boundaries and API contracts.
- Use the testing pages to create test cases before the final demonstration.
- Use the documentation pages as the basis for the final project report and presentation.
- Use the diagram pages to redraw professional diagrams with standard UML/flowchart notation.

Table of Contents

Section 3

No.	Section	Pages
1	Project Overview and Scope	4-6
2	Brainstorming and Ideation	7-10
3	Requirement Analysis	11-16
4	Project Design	17-23
5	Project Planning	24-27
6	Development Pre-Requisites	28-34
7	Testing and Quality Assurance	35-39
8	Documentation	40-42
9	Demonstration and Presentation	43-45
10	Images and Diagram Specifications	46-48
11	Final Checklist and Conclusion	49-50

Note: Page ranges refer to the logical page sequence of this generated report. The exact printed pagination may vary slightly if the PDF viewer applies different font rendering.



INVALID VALID
↓ ↓
Error JSON [Template Rules]
↓
[AI / Draft Service]
↓
[Output Validation]
↓
[Preview + Persistence]
↓
[Renderer]
↓
PDF / DOCX

SEQUENCE
User → UI: enter document details
UI → API: POST /api/documents/generate
API → Validator: validate request
Validator → API: valid / invalid
API → Template Repository: load template
API → AI Adapter: generate structured draft
AI Adapter → Provider: model request
Provider → AI Adapter: response
AI Adapter → Draft Validator: normalized content
Draft Validator → Renderer: approved structure
Renderer → API: preview/export result
API → UI: success response

FULL ACTIVITY DIAGRAM
START
↓
Select Document Type
↓
Enter Required Details
↓
Validate Input

└─ INVALID → Show Errors → Correct Input → Validate Input
↓ VALID
Load Template + Rules
↓
Generate Draft
↓
Validate Generated Output
└─ INVALID → Retry/Regenerate or Show Error
↓ VALID
Preview Draft
↓
User Review
└─ EDIT → Update Input → Generate Draft
↓ APPROVE
Export PDF/DOCX
↓
END

FULL ER DIAGRAM
TEMPLATE ————— DRAFT
template_id PK draft_id PK
name template_id FK
version input_json
schema_json output_text
1
N
GENERATION_RUN
run_id PK
draft_id FK
provider
model
status

OPTIONAL:
_____ 1 N _____
USER _____ DRAFT
user_id PK user_id FK

Project Overview: Problem Statement

Section 4

Legal and administrative documents often require repetitive fields, formal structure, consistent language, and careful review. Manual drafting can be slow and may introduce formatting inconsistencies.

LegalEase is proposed as an AI-powered document generator that guides a user through a structured form, validates inputs, prepares a draft, and presents the result for review and export.

The project problem is therefore not simply text generation. It is the controlled transformation of user requirements into a structured document while preserving traceability, predictable formatting, privacy protections, and human review.

The system should separate user data collection, document rules, AI assistance, rendering, storage, and export. This separation makes the application easier to test and maintain.

The initial scope should focus on a small set of clearly defined document types. Additional templates can be added after the core workflow is stable.

Success is measured through functional completeness, correctness of required fields, response reliability, usability, document formatting, and test coverage rather than by claiming that generated text is legally authoritative.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Overview: Objectives and Scope

Section 5

Primary objective: build a web-based prototype that generates structured legal-document drafts from user inputs.

Secondary objective: provide a clean API layer that can be tested independently of the user interface.

Create reusable document templates and field definitions instead of hard-coding every document separately.

Implement input validation for required fields, formats, lengths, and acceptable values.

Provide an AI service abstraction so the project can change model providers without rewriting business logic.

Store only the data necessary for the chosen prototype workflow and apply appropriate access controls.

Generate a readable preview and export-ready document.

Provide clear status messages for validation errors, generation failures, and successful output.

Include a testing suite covering normal, boundary, invalid, and failure scenarios.

Prepare a demonstration script that shows the complete user journey from input to final draft.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Pre-Requisite Categories

Section 6

Knowledge prerequisites include Python fundamentals, REST APIs, JSON, HTML/CSS/JavaScript, databases, Git, software testing, and basic prompt engineering.

Technical prerequisites include a development machine, Python runtime, package manager, code editor, Git, API testing tool, browser, and optional database client.

Project prerequisites include a defined scope, document templates, sample test data, UI wireframes, API contracts, and acceptance criteria.

Security prerequisites include environment-variable management, secret protection, input sanitization, authentication requirements, logging policy, and data-retention decisions.

Quality prerequisites include coding standards, branch strategy, test strategy, issue tracking, review process, and release checklist.

Demonstration prerequisites include sample inputs, prepared output examples, screenshots, diagrams, and a short explanation of limitations.

Legal-domain prerequisite: the team should define the intended jurisdiction and document scope, avoid unsupported legal claims, and make clear that generated drafts require appropriate review.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Brainstorming and Ideation: Goals

Section 7

- Brainstorming starts by identifying the user problem rather than immediately selecting a technology.
- List common document-preparation pain points: repeated data entry, inconsistent formatting, forgotten fields, slow drafting, and difficulty maintaining templates.
- Identify target users for the academic prototype and write representative user stories.
- Separate must-have features from optional ideas. This prevents an ambitious feature list from delaying the core deliverable.
- Explore alternative workflows: guided form generation, conversational collection, template selection, clause library, document upload, and export.
- Record assumptions explicitly. An assumption is not the same as a confirmed requirement.
- Capture constraints such as time, available infrastructure, model access, privacy, and document complexity.
- End ideation with a short product concept that the team can explain in one minute.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Brainstorming: Feature Ideation

Section 8

- Feature 1: document-type selection with descriptions and required-field lists.
- Feature 2: guided form that shows only relevant fields.
- Feature 3: real-time validation and clear error messages.
- Feature 4: AI-assisted drafting using a controlled prompt template.
- Feature 5: preview with headings, clauses, parties, dates, and signatures.
- Feature 6: regenerate or revise a section without losing unrelated information.
- Feature 7: export to PDF or DOCX through a dedicated rendering layer.
- Feature 8: draft history for a prototype, with deletion and retention controls.
- Feature 9: administrator-managed templates for future expansion.
- Feature 10: audit-friendly generation metadata such as template version and timestamp.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Brainstorming: User Stories

Section 9

- As a user, I want to choose a document type so that I can provide the correct information.
- As a user, I want required fields to be identified so that I do not miss important information.
- As a user, I want validation errors explained so that I can correct them without guessing.
- As a user, I want to preview the generated document before exporting it.
- As a user, I want the document to have consistent headings, spacing, and signature areas.
- As a user, I want to revise inputs and regenerate a draft.
- As a developer, I want document rules separated from the API so that templates can evolve independently.
- As a tester, I want deterministic mock AI responses so that API tests do not depend on an external model.
- As a project reviewer, I want diagrams and test evidence so that the implementation can be evaluated systematically.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Ideation Output: Product Vision and Boundaries

Section 10

Product vision: LegalEase helps users prepare structured draft documents through a guided, validated, AI-assisted workflow.

In scope: template selection, data collection, validation, draft generation, preview, export, error handling, testing, and documentation.

Potentially out of scope for the first version: automated legal research, court filing, legal representation, guaranteed legal correctness, autonomous legal advice, and unrestricted document generation.

Each generated document should carry appropriate draft/review messaging.

The AI component should be treated as an assistive service. Critical facts should originate from user inputs or verified templates rather than being invented by the model.

Versioning is important. A document should be associated with the template version and, where useful, model configuration used to produce it.

The ideation phase ends with a prioritized backlog and an agreed MVP.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Requirement Analysis: Functional Requirements

Section 11

- FR-01: The system shall allow the user to select an available document type.
- FR-02: The system shall present fields required for the selected document.
- FR-03: The system shall validate mandatory values before generation.
- FR-04: The system shall normalize suitable input formats such as dates and whitespace.
- FR-05: The system shall construct a structured generation request.
- FR-06: The system shall invoke an AI/document-generation service through a defined interface.
- FR-07: The system shall return a structured draft or a controlled error.
- FR-08: The system shall render a readable preview.
- FR-09: The system shall support export through the chosen document renderer.
- FR-10: The system shall provide meaningful status and validation messages.
- FR-11: The system shall support template version identification.
- FR-12: The system shall expose health/status information for deployment checks.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Requirement Analysis: Non-Functional Requirements

Section 12

Performance: ordinary validation requests should return promptly; AI latency should be displayed or handled asynchronously when necessary.

Reliability: failures from external services should be handled without exposing internal stack traces.

Security: secrets must not be stored in source code; inputs must be validated and outputs treated as untrusted until reviewed.

Privacy: collect the minimum data needed for the selected workflow and define retention behavior.

Maintainability: modules should have clear responsibilities and tests.

Scalability: the AI provider and database should be replaceable without redesigning the UI.

Usability: forms should be understandable, keyboard accessible, and consistent.

Observability: logs should support troubleshooting while avoiding unnecessary sensitive content.

Portability: development should be reproducible through a requirements file and documented setup steps.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Requirement Analysis: Use Cases

Section 13

- UC-01 Select Document Type: user views available templates and selects one.
- UC-02 Enter Details: user supplies parties, dates, descriptions, and other template-specific fields.
- UC-03 Validate: server checks schema and business rules.
- UC-04 Generate Draft: service combines validated data, template rules, and AI assistance.
- UC-05 Review Draft: user examines generated content and identifies changes.
- UC-06 Regenerate: user modifies inputs or selected sections and requests another draft.
- UC-07 Export: user downloads the approved draft in the supported format.
- UC-08 Error Recovery: user receives an actionable message when a dependency fails.
- UC-09 Health Check: operator confirms that the application and dependencies are reachable.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Requirement Analysis: Data Requirements

Section 14

DocumentTemplate: id, name, version, field definitions, rules, and rendering configuration.

DocumentRequest: template id, validated user fields, optional generation instructions, and request metadata.

DocumentDraft: generated content, template version, status, timestamps, and optional revision information.

User/session information: only what is required by the selected authentication design.

Generation metadata: provider/model identifier, request time, latency, and error status where appropriate.

Audit metadata: events needed for troubleshooting and project evaluation, without unnecessary sensitive content.

Export metadata: output format, creation time, and file status.

Configuration: provider endpoint, model setting, database URL, and other non-code configuration values supplied through environment variables.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Requirement Analysis: API Requirements

Section 15

- GET /health — confirms the service is running.
 - GET /api/templates — lists supported document templates.
 - GET /api/templates/{id} — returns a template definition.
 - POST /api/documents/validate — validates user-provided fields.
 - POST /api/documents/generate — creates a draft through the generation service.
 - GET /api/documents/{id} — retrieves a stored draft when persistence is enabled.
 - POST /api/documents/{id}/export — renders a selected output format.
 - DELETE /api/documents/{id} — removes a draft when deletion is supported.
- API responses should use consistent JSON structures with success data, error codes, messages, and validation details.
- OpenAPI documentation generated by FastAPI should be reviewed and kept synchronized with implementation.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Requirement Analysis: Acceptance Criteria

Section 16

- A template can be selected and its fields are displayed correctly.
- Submitting incomplete required fields produces validation errors without calling the AI service.
- Valid sample data produces a structured draft using the selected template.
- The generated document contains the expected major sections and user-supplied facts.
- An AI timeout or provider error returns a controlled application error.
- Export produces a readable file in the supported format.
- API documentation exposes the implemented endpoints and schemas.
- Automated tests pass in a clean development environment.
- Sensitive configuration is loaded from environment variables rather than committed source.
- The demonstration can be repeated using prepared test data without manual database repair.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Design: High-Level Architecture

Section 17

Use a layered architecture to isolate presentation, API, business rules, AI integration, persistence, and document rendering.

The frontend collects inputs and displays results. It should not contain provider secrets or trusted business rules.

The FastAPI layer exposes typed REST endpoints and coordinates validation and service calls.

The service layer implements document-generation workflows and isolates business logic from HTTP details.

The AI adapter translates a structured generation request into a provider-specific request and normalizes the response.

The repository layer handles database access where persistence is required.

The rendering layer transforms structured drafts into PDF/DOCX/HTML outputs.

Configuration is loaded once through a typed settings module and injected into dependent services.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Design: Architecture Diagram

Section 18

Recommended flow: User → Web UI → FastAPI Router → Validation Service → Document Service → AI Adapter → Draft Validator → Renderer → Output.

Cross-cutting controls: authentication, authorization, logging, error handling, rate limiting, and configuration management.

External dependency boundary: AI provider should be accessed only through the adapter interface.

Storage boundary: application code should use repository interfaces rather than embedding SQL throughout route handlers.

Project Design: Module Structure

Section 19

- app/main.py: creates the FastAPI application and registers routers.
- app/api/: HTTP route definitions and dependency wiring.
- app/schemas/: Pydantic request/response models.
- app/services/: document workflow, validation, generation, and export orchestration.
- app/ai/: provider adapter, prompt builder, and response normalization.
- app/templates/: versioned document templates and field definitions.
- app/repositories/: database access abstractions.
- app/rendering/: PDF/DOCX/HTML rendering utilities.
- tests/: unit, API, integration, and regression tests.
- config/: environment configuration examples and non-secret project settings.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Design: Database Design

Section 20

Use relational tables when the prototype needs persistent templates, drafts, users, or audit events.

templates table: template_id, name, version, status, schema_json, created_at.

drafts table: draft_id, template_id, input_json, output_text, status, created_at, updated_at.

generation_runs table: run_id, draft_id, provider, model, duration_ms, status, error_code.

users table is optional for an anonymous prototype and should be added only when authentication is required.

Indexes should support common lookups such as draft_id, template_id, and timestamps.

Sensitive fields should be minimized and protected according to the deployment environment.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Design: Data Flow Diagram

Section 21

The data flow begins with user input and ends with a reviewed document.

Validation is a gate: invalid requests stop before generation.

The generation service receives only the fields required by the selected template.

The AI response is normalized and checked before it is shown as a final draft.

Export uses the structured draft rather than directly trusting arbitrary model markup.

Project Design: Sequence Diagram

Section 22

The sequence should be deterministic around validation and error handling even if the AI output itself is probabilistic.

Client submits a request; API validates schema; service loads template; prompt builder constructs controlled instructions; adapter calls provider; service validates output; renderer creates preview/export.

Any dependency failure should move to an explicit error path and must not expose credentials or internal stack traces.

Project Design: Security and Privacy

Section 23

Keep API keys and database credentials outside source control.

Use HTTPS in deployed environments and avoid sending sensitive data to external providers unless the project design explicitly permits it.

Apply authentication and authorization if documents are associated with accounts.

Validate file uploads if enabled, restrict size/type, and scan or safely process untrusted content.

Prevent prompt injection from untrusted document content by separating system instructions, user data, and retrieved material.

Do not log complete legal documents by default. Prefer identifiers, status codes, and redacted diagnostics.

Define retention and deletion behavior before enabling persistent storage.

Use least-privilege credentials for database and external service access.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Planning: Work Breakdown Structure

Section 24

- Phase 1: problem definition and ideation.
- Phase 2: requirements and acceptance criteria.
- Phase 3: UI, API, database, and architecture design.
- Phase 4: development environment setup.
- Phase 5: core API and validation implementation.
- Phase 6: template and generation service implementation.
- Phase 7: AI adapter integration and mock provider.
- Phase 8: rendering/export and persistence.
- Phase 9: testing, defect correction, and security review.
- Phase 10: documentation, screenshots, diagrams, and final demonstration.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Planning: Suggested Timeline

Section 25

- Week 1: define scope, personas, user stories, document types, and project risks.
- Week 2: finalize requirements, wireframes, architecture, schemas, and API contracts.
- Week 3: configure repository, Python environment, FastAPI skeleton, health endpoint, and CI/test command.
- Week 4: implement templates, validation, and core document service.
- Week 5: implement AI adapter with mock mode and provider integration.
- Week 6: implement persistence, preview, and export.
- Week 7: perform unit, API, integration, security, and usability testing.
- Week 8: fix defects, prepare screenshots/diagrams, complete documentation, rehearse demonstration.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Planning: Risk Register

Section 26

Risk: AI produces unsupported content. Mitigation: structured prompts, template constraints, output validation, and human review.

Risk: external AI service unavailable. Mitigation: mock provider and graceful error handling.

Risk: requirements expand. Mitigation: freeze MVP scope and manage enhancements as backlog items.

Risk: sensitive data exposure. Mitigation: minimum-data design, secret management, logging controls, and access restrictions.

Risk: export formatting defects. Mitigation: sample documents, rendering tests, and visual inspection.

Risk: environment differences. Mitigation: pinned dependencies, setup documentation, and reproducible commands.

Risk: insufficient test time. Mitigation: write high-value tests alongside development.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Project Planning: Team Roles and Deliverables

Section 27

Project lead: scope, schedule, integration, and final review.

Backend developer: FastAPI routes, schemas, services, database, and tests.

Frontend developer: forms, validation display, preview, and responsive layout.

AI integration developer: prompt construction, adapter, provider configuration, and mock mode.

QA/documentation member: test plan, defect log, screenshots, diagrams, and report.

For a solo academic project, these roles can be treated as responsibility areas rather than separate people.

Primary deliverables: source repository, environment file template, API documentation, test suite, sample outputs, diagrams, final report, and demonstration presentation.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Development: Environment Prerequisites

Section 28

- Install a supported Python version appropriate to the chosen FastAPI dependency set.
- Create an isolated virtual environment for the project.
- Install dependencies from a requirements.txt or pyproject.toml file.
- Install Git and configure a project repository.
- Use an editor with Python support, linting, formatting, and debugging.
- Install an API client such as the browser's OpenAPI UI, curl, or another REST client.
- Install a database tool if persistent storage is enabled.
- Create a .env file for local secrets and a .env.example containing variable names only.
- Document the exact setup commands so another evaluator can reproduce the environment.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Development: Suggested Python/FastAPI Stack

Section 29

- Python: primary backend language.
- FastAPI: REST API framework with automatic OpenAPI documentation.
- Pydantic: typed request and response validation.
- Uvicorn: ASGI development server.
- SQLAlchemy or another selected data-access layer: persistence abstraction.
- SQLite: simple prototype database; PostgreSQL: stronger production-oriented option.
- Jinja2 or a selected template engine: HTML/document templating where needed.
- ReportLab, python-docx, or another approved renderer: document export depending on requirements.
- Pytest: automated testing.
- Ruff/Black or an agreed equivalent: formatting and code-quality consistency.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Development: Project Initialization

Section 30

Create the repository and establish a clean directory structure.

Add a README with purpose, setup, environment variables, run command, test command, and limitations.

Create the FastAPI application entry point.

Add a /health endpoint.

Create a versioned API prefix such as /api.

Define the first Pydantic schemas.

Configure dependency injection for services.

Add a test that imports the application and checks the health endpoint.

Commit the initial working skeleton before adding complex features.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Development: Core Workflow

Section 31

- Step 1: receive a document-generation request.
- Step 2: validate the selected template identifier.
- Step 3: validate required and optional fields.
- Step 4: normalize safe formats.
- Step 5: load template rules and generation instructions.
- Step 6: build a structured prompt or deterministic template request.
- Step 7: call the AI adapter or mock service.
- Step 8: normalize and validate the generated result.
- Step 9: persist the draft when enabled.
- Step 10: return preview data and export metadata.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Development: AI Integration Prerequisites

Section 32

- Define the AI adapter interface before selecting a specific provider implementation.
- Keep provider credentials in environment variables.
- Use structured prompts with explicit output requirements.
- Ask the model to use only supplied facts for factual fields.
- Use schema-constrained output where supported, and validate the response again in application code.
- Set reasonable token and timeout limits.
- Implement retry behavior only where safe and bounded.
- Create a mock AI provider that returns deterministic fixtures for automated tests.
- Record model/provider metadata for reproducibility where policy and privacy permit.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Development: Prompt and Template Strategy

Section 33

Templates define document structure; prompts should assist generation rather than replace deterministic business rules.

Include the document type, template version, user-provided facts, required sections, prohibited assumptions, and output schema.

Explicitly instruct the model not to invent names, dates, amounts, addresses, legal citations, or events that are not provided.

Separate instructions from user data using clear delimiters.

Require missing information to be represented as a validation request rather than fabricated content.

Version prompts alongside templates so changes can be tested and explained.

Maintain sample inputs and expected structural outputs for regression testing.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Development: Error Handling and Observability

Section 34

Return 4xx responses for client validation problems and appropriate 5xx responses for unexpected server failures.

Use application-level error codes such as `TEMPLATE_NOT_FOUND`, `VALIDATION_FAILED`, `AI_TIMEOUT`, `AI_PROVIDER_ERROR`, and `EXPORT_FAILED`.

Do not return stack traces, API keys, provider headers, or database credentials to clients.

Log correlation/request IDs to connect API events during debugging.

Measure generation latency and dependency failure counts.

Use health checks that distinguish application availability from dependency availability where appropriate.

Create a clear operator message for configuration errors such as missing required environment variables.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Testing: Test Strategy

Section 35

Testing begins before the final build. Every requirement should map to one or more test cases.

Unit tests cover validators, prompt builders, template selection, output normalization, and rendering helpers.

API tests cover status codes, request schemas, response schemas, and error handling.

Integration tests cover database access and the generation service using a mock AI provider.

End-to-end tests cover the complete UI/API workflow where practical.

Security tests cover authentication boundaries, input limits, secret handling, and unsafe file behavior.

Visual tests or manual inspection cover exported document layout.

Regression tests protect previously fixed defects.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Testing: Sample Test Cases

Section 36

- TC-01 valid template selection → expected success.
- TC-02 unknown template ID → expected controlled not-found error.
- TC-03 missing required field → expected validation error.
- TC-04 invalid date format → expected validation error.
- TC-05 excessive field length → expected rejection or defined truncation rule.
- TC-06 mock AI success → expected structured draft.
- TC-07 AI timeout → expected controlled dependency error.
- TC-08 malformed AI response → expected output-validation failure.
- TC-09 export success → expected file with required headings.
- TC-10 unauthorized access → expected denial when authentication is enabled.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Testing: Boundary and Negative Cases

Section 37

Empty strings, whitespace-only values, very long text, unexpected Unicode, and malformed JSON should be considered.

Dates should be tested around boundary conditions relevant to the document type.

Numeric fields should reject invalid signs, precision, or ranges where applicable.

Template version mismatches should produce a controlled result.

Database unavailable scenarios should not crash the API process.

AI provider returning partial output should be detected.

Repeated generation requests should be evaluated for idempotency or explicit revision semantics.

Large document export should be tested for memory and timeout behavior.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Testing: Security and Privacy Verification

Section 38

- Verify that no secrets are committed to Git.
- Search logs and error responses for accidental sensitive-data exposure.
- Verify request size limits and safe handling of user-controlled strings.
- Verify authentication and authorization for every protected resource.
- Verify that a user cannot retrieve another user's draft when multi-user persistence is enabled.
- Verify deletion behavior and retention assumptions.
- Verify that uploaded files, if supported, cannot cause arbitrary file access.
- Verify that prompt-injection content is treated as data and cannot override application rules.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Testing: Defect Management and Release Gate

Section 39

Record each defect with ID, severity, steps to reproduce, expected result, actual result, environment, and status.

Prioritize defects that block core workflows, expose data, corrupt output, or prevent demonstration.

Before release, run the full automated suite in a clean environment.

Verify that documentation commands match the actual project.

Verify that the demonstration data produces the expected result.

Verify that exported files open correctly on the target evaluation device.

Release only when critical defects are resolved or explicitly documented as limitations.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Documentation: Technical Documentation

Section 40

README: project purpose, architecture summary, setup, configuration, run commands, tests, and limitations.

API documentation: endpoints, parameters, schemas, status codes, example requests, and example responses.

Architecture document: component responsibilities, data flow, deployment assumptions, and external dependencies.

Database document: tables, fields, relationships, indexes, and retention considerations.

AI document: provider interface, prompt strategy, model settings, mock mode, and validation.

Testing document: strategy, test cases, results, known limitations, and defect history.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Documentation: User Guide

Section 41

- Explain how to open the application and select a document type.
- Explain every required field with an example where appropriate.
- Explain validation messages and how to correct them.
- Explain that generated output is a draft requiring appropriate review.
- Explain preview and export actions.
- Explain how to revise inputs and regenerate.
- Provide troubleshooting steps for common configuration or service failures.
- Use screenshots with numbered callouts when the final UI is stable.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Documentation: Project Report Evidence

Section 42

Include problem statement, objectives, scope, methodology, architecture, requirements, implementation, testing, results, limitations, and future work.

Capture screenshots of the home page, document selection, input form, validation message, generation state, preview, and export result.

Include diagrams: use case, architecture, data flow, sequence, database ER model, deployment, and testing flow.

Include sample API requests/responses with non-sensitive data.

Include test-result tables and explain any failed or skipped tests.

Maintain a version/date on major report revisions so the evaluator can identify the final build.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Demonstration: Recommended Flow

Section 43

- Start with a one-minute explanation of the problem and the project's goal.
- Show the architecture diagram before opening the application so the evaluator understands the components.
- Select a document type.
- Enter a prepared, realistic but non-sensitive sample dataset.
- Trigger validation and show that required fields are checked.
- Generate the draft and explain the role of the AI service.
- Review the output and point out how user-provided facts appear in the document.
- Export the document and open the resulting file.
- Briefly show API documentation and automated test results.
- End with limitations, future enhancements, and questions.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Demonstration: Evidence Checklist

Section 44

Application launches successfully on the demonstration machine.

Health endpoint responds successfully.

Frontend form is functional.

At least one complete document template works.

Validation error can be intentionally demonstrated.

Successful generation can be demonstrated.

Export file can be opened.

API documentation is accessible.

Automated tests have a recent successful run.

Diagrams and screenshots correspond to the implemented version.

All sample data is fictional or non-sensitive.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Demonstration: Common Questions and Answers

Section 45

Q: Is the output legally guaranteed? A: No. It is an AI-assisted draft and should be reviewed appropriately.

Q: Why use FastAPI? A: It provides a modern Python API framework, typed validation through Pydantic, and automatic OpenAPI documentation.

Q: What happens if the AI service fails? A: The adapter returns a controlled error and the application can use a mock service during development/testing.

Q: How is hallucination reduced? A: Controlled templates, explicit input boundaries, structured output, validation, and human review reduce—but cannot eliminate—risk.

Q: Can another AI provider be used? A: Yes, if it implements the adapter interface.

Q: Can more document types be added? A: Yes, by adding versioned templates, schemas, validation rules, tests, and rendering rules.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Images: Required Screenshot Plan

Section 46

Image 1: project landing page with LegalEase title and document-type selection.

Image 2: document selection screen showing available templates.

Image 3: input form with required fields and helper text.

Image 4: validation error state with a missing required field.

Image 5: loading/generation state.

Image 6: generated-document preview.

Image 7: export confirmation and file output.

Image 8: FastAPI interactive API documentation.

Image 9: test execution result in the terminal or CI interface.

Image 10: project folder structure in the code editor.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Full Diagrams: UML / Process Set

Section 47

Recommended diagram set includes Use Case Diagram, Activity Diagram, Component Diagram, Sequence Diagram, Data Flow Diagram, ER Diagram, Deployment Diagram, and Test Flow Diagram.

Use Case actors: User, Administrator (optional), AI Provider, Document Renderer.

Activity flow: select template → enter details → validate → generate → validate output → review → export.

Component diagram: Frontend, API, Validation Service, Document Service, AI Adapter, Repository, Renderer, Database, AI Provider.

Deployment diagram: browser/device → web/API server → database and external AI provider.

Each diagram should include a title, legend if needed, version/date, and only components that actually exist in the implementation.

Full Diagrams: ER and Deployment

Section 48

ER model should represent templates, drafts, generation runs, and users only when persistence/authentication is implemented.

Relationship: Template 1→N Drafts; Draft 1→N GenerationRuns; User 1→N Drafts when users are enabled.

Deployment should show the browser, application server, database, and external AI provider with trust boundaries.

Use arrows to identify HTTP/HTTPS, database protocol, and AI-provider API calls.

Do not include infrastructure components that are only hypothetical. Label optional components clearly.

Final Pre-Requisite Checklist

Section 49

- ☐ Project title and problem statement finalized.
- ☐ MVP document types selected.
- ☐ User stories and functional requirements approved.
- ☐ Non-functional requirements documented.
- ☐ Architecture and data flow designed.
- ☐ API endpoints and schemas agreed.
- ☐ Development environment installed.
- ☐ Repository initialized and setup documented.
- ☐ Secrets/configuration strategy established.
- ☐ Template and prompt versioning strategy established.
- ☐ Mock AI provider available.
- ☐ Database strategy selected.
- ☐ Export strategy selected.
- ☐ Unit and API tests created.
- ☐ Security/privacy checks planned.
- ☐ Screenshots captured from the final build.
- ☐ Full diagrams redrawn neatly for submission.
- ☐ Demonstration dataset prepared.
- ☐ Final report reviewed for consistency.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.

Conclusion and Future Enhancement Plan

Section 50

LegalEase can be implemented successfully when the team treats AI generation as one controlled component inside a broader software system.

The most important prerequisites are clear scope, typed requirements, modular architecture, reliable validation, provider abstraction, secure configuration, repeatable testing, and evidence-based documentation.

Future versions can add more templates, multilingual interfaces, document comparison, richer clause libraries, role-based administration, improved audit trails, and enterprise deployment controls.

Future AI improvements may include structured extraction, retrieval from approved sources, stronger output schemas, and human-in-the-loop review workflows.

Any expansion into legal research, jurisdiction-specific advice, filing, or automated legal decisions requires additional domain validation, governance, privacy review, and professional oversight.

For an academic project, a small but complete workflow is preferable to many incomplete features. A stable end-to-end demonstration provides clear evidence that the architecture and requirements were translated into working software.

Final submission package should contain the source code, setup instructions, requirements, test evidence, screenshots, diagrams, sample outputs, and this report.

End of report.

PAGE OUTPUT / REVIEW CHECKPOINT
Inputs: requirements, assumptions, sample data, and project constraints.
Process: apply the relevant design/development/testing/documentation activity.
Output: an artifact that can be reviewed, tested, or demonstrated.
Review question: can another team member reproduce or verify this result?
Evidence: save the related source file, screenshot, test result, or diagram in the project repository.